AirSunBox
MONITOREO AMBIENTAL

Guía de despliegue y Runbook

Sistema de monitoreo de calidad del aire en tiempo real

Dany de León Santizo
Juan Carlos Rodríguez López

Martes, 27 de octubre, 2026
Quetzaltenango

Índice

1. Referencia rápida....3

2. Despliegue normal (automático)....4

2.1 Verificación posterior al despliegue....4
2.2 Cambios que requieren un paso previo....5

3. Instalación desde cero....6

4. Migraciones de base de datos....7

5. Rollback....8

5.1 Código (caso normal)....8
5.2 Base de datos....8
5.3 Disco completo....9
5.4 Secretos....9

6. Monitoreo....10

6.1 Chequeos....10
6.2 Frecuencia sugerida....10

7. Atención de incidentes....11

I-01 · La web no responde (código 000, 502 o 504)....11
I-02 · El backend no arranca o queda unhealthy....11
I-03 · Base de datos o Redis caídos, o disco lleno....12
I-04 · El navegador avisa que el certificado no es válido....12
I-05 · No llegan los correos (2FA o recuperación de contraseña)....13
I-06 · Estaciones sin datos o "0 estaciones activas"....13
I-07 · El job deploy falla....13
I-08 · Sospecha de un secreto filtrado....14
I-09 · Muchos usuarios ven "Tu contraseña cambió" o quedan fuera de sesión....14
I-10 · Respuestas 429 (*Too Many Requests*)....14
I-11 · La instancia se detuvo o cambió la IP....15

8. Mantenimiento....16

9. Registro de cambios operativos....17

1. Referencia rápida

Elemento	Valor
URL	https://18.223.121.208
Instancia	EC2 i-07a637df8369c3f6c, us-east-2c, Ubuntu 24.04 — **sin IP elástica**: detenida fuera de los periodos de demostración para ahorrar créditos; al encenderla cambia la IP (ver I-11)
Acceso SSH	alias airsunbox-ec2 (usuario ubuntu, puerto 22 abierto solo a 2 IPs)
Carpeta del proyecto	/home/ubuntu/apps/airsunbox/plataforma-iot-airsunbox (en adelante, $APP)
Compose	sudo docker compose -p airsunbox -f docker-compose.yml … (ejecutado dentro de $APP)
Contenedores	iot_frontend, iot_backend, iot_db, iot_redis, iot_mqtt
Secretos	SSM Parameter Store /airsunbox/prod/* (cuenta 009644552142, us-east-2)
CI/CD	GitHub Actions, workflow CI (jobs backend, frontend, deploy)
Responsables	Producto y aplicación: JCR2005 · Infraestructura AWS y CD: DNLS89

En los comandos de esta guía:

```
APP=/home/ubuntu/apps/airsunbox/plataforma-iot-airsunbox
```

Para operar AWS desde una terminal: AWS CLI v2 y `aws login --region us-east-2`.

2. Despliegue normal (automático)

El despliegue a producción **no tiene pasos manuales**: lo dispara el merge a `main`.

1. Crear una rama desde `develop`: `feature/<tema>`, `fix/<tema>` o `docs/<tema>`.
2. Abrir un PR hacia `develop`. El CI corre:
   - migraciones sobre una base vacía;
   - typecheck y build del backend;
   - pruebas de humo;
   - build del frontend.
3. Con el CI en verde, hacer merge a `develop`.
4. Abrir el PR de *release* `develop → main`. La regla **Proteger main** exige el PR y los checks `backend` y `frontend` en verde.
5. Al hacer merge a `main`, el job `deploy`:
   - asume el rol `airsunbox-github-deploy` por OIDC;
   - con SSM Run Command, avanza el servidor al commit probado (`git merge --ff-only`);
   - reconstruye y levanta solo `backend` y `frontend` (la base, Redis y MQTT no se reinician);
   - verifica backend `healthy`, frontend `running` y HTTPS 200.
6. Revisar en *Actions* que `deploy` termine en verde.

El job de despliegue falla si:

- el servidor no está en la rama `main`;
- el servidor tiene cambios sin commitear (`git status` debe estar limpio);
- el commit de `origin/main` no coincide con el probado.

Los archivos propios del servidor (`backend/.env`) están en `.gitignore`.

2.1 Verificación posterior al despliegue

```
curl -s -o /dev/null -w '%{http_code}\n' https://18.223.121.208/          # 200
curl -s https://18.223.121.208/api/public/reports/stats                    # JSON con totales
ssh airsunbox-ec2 "sudo docker ps --format '{{.Names}}\t{{.Status}}'"      # 5 contenedores
Up, healthy
ssh airsunbox-ec2 "sudo docker logs --since 10m iot_backend 2>&1 | grep -c '\"level\":50'"
# 0 errores
```

2.2 Cambios que requieren un paso previo

Cambio	Paso previo al merge a `main`
Migración de base de datos nueva	Aplicarla en producción (sección 4): el job `deploy` no ejecuta migraciones
Secreto nuevo	Crearlo en SSM y agregarlo a `SECRET_NAMES` en `backend/src/lib/secrets.ts`
Variable de entorno no secreta	Agregarla al `backend/.env` del servidor y a `backend/.env.example`

3. Instalación desde cero

Para levantar la plataforma en una instancia nueva (recuperación ante desastre).

1. **Infraestructura:**
   - EC2 Ubuntu 24.04 con el rol `sanik-iot-ec2-role` y disco cifrado (el cifrado por defecto ya está activo en la región);
   - security group con 80 y 443 abiertos y 22 restringido.

   La definición está en `terraform/`.
2. **Software:** instalar Docker Engine con el plugin compose y `git`.
3. **Código:** clonar el repositorio en `/home/ubuntu/apps/airsunbox` y quedar en la rama `main`.
4. **Configuración:** crear `$APP/backend/.env` a partir de `backend/.env.example`, sin secretos, con:
   - `SSM_SECRETS_PATH=/airsunbox/prod` y `AWS_REGION=us-east-2`;
   - `FRONTEND_URL` y `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_FROM`;
   - permisos `chmod 600`.
5. **Contraseña inicial de la base:** crear `$APP/.env` con `DB_PASSWORD=<valor de /airsunbox/prod/DB_PASSWORD>`. Postgres lo usa solo al crear la base.
6. **Certificado:**
   - emitirlo con certbot en modo *webroot* (`/var/www/certbot`);
   - crear el *hook* `/etc/letsencrypt/renewal-hooks/deploy/recargar-nginx.sh` con `docker exec iot_frontend nginx -s reload`.
7. **Arranque:** `sudo docker compose -p airsunbox -f docker-compose.yml up -d --build`. Con el volumen vacío, Postgres aplica el esquema y todas las migraciones (`database/initdb/00-schema.sh`).
8. **Datos:** restaurar el último respaldo (sección 5.2) o crear el primer superadmin con `node dist/scripts/createSuperadmin.js` dentro de `iot_backend`.
9. Verificar como en la sección 2.1.

4. Migraciones de base de datos

Crear una migración:

1. Crear `database/vNN-descripcion.sql` **idempotente** (`IF NOT EXISTS`, `ON CONFLICT DO NOTHING`, `ADD COLUMN IF NOT EXISTS`).
2. Agregarla al final de la lista de `database/initdb/00-schema.sh`.
3. El CI la aplica sobre una base vacía en cada PR: si falla, el PR queda rojo.

Aplicarla en producción, antes del merge a `main`:

```
ssh airsunbox-ec2 "sudo docker exec -i iot_db psql -U iot_user -d iot_platform -v
ON_ERROR_STOP=1" \
  < plataforma-iot-airsunbox/database/vNN-descripcion.sql
```

Antes de una migración que modifica o borra datos, hacer un respaldo (sección 5.2).

5. Rollback

5.1 Código (caso normal)

1. En GitHub, revertir el merge problemático con un PR hacia `develop` (`git revert -m 1 <commit de merge>`), con el CI en verde.
2. *Release* a `main`: el job `deploy` despliega la versión anterior.

   Tiempo estimado: 10–15 minutos. No requiere acceso al servidor.

Emergencia (la web está caída y no se puede esperar el CI): en el servidor, volver al commit anterior **sin dejar cambios locales**, para que el próximo despliegue pueda avanzar:

```
ssh airsunbox-ec2 "cd /home/ubuntu/apps/airsunbox && git log --oneline -5"   # elegir el
commit bueno
ssh airsunbox-ec2 "cd /home/ubuntu/apps/airsunbox && git checkout -B main <commit-bueno> && \
  cd plataforma-iot-airsunbox && sudo docker compose -p airsunbox -f docker-compose.yml up -d
--build --no-deps backend frontend"
```

Después hacer el revert por PR (paso 1): el siguiente `deploy` vuelve a dejar el servidor alineado con `origin/main`.

5.2 Base de datos

Respaldo:

```
ssh airsunbox-ec2 "sudo docker exec iot_db pg_dump -U iot_user -d iot_platform -Fc" >
airsunbox-$(date +%F).dump
```

`pg_dump` avisa de *circular foreign-key constraints* en `continuous_agg`: es normal en TimescaleDB y el respaldo es válido.

Restauración. Se recrea la base y se sigue el procedimiento de TimescaleDB (`timescaledb_pre_restore` / `timescaledb_post_restore`). Primero se detiene el backend, para que no escriba mientras tanto:

```
ssh airsunbox-ec2 "sudo docker stop iot_backend"
ssh airsunbox-ec2 "sudo docker exec iot_db psql -U iot_user -d postgres \
  -c 'DROP DATABASE iot_platform WITH (FORCE)' -c 'CREATE DATABASE iot_platform'"
ssh airsunbox-ec2 "sudo docker exec iot_db psql -U iot_user -d iot_platform \
  -c 'ALTER EXTENSION timescaledb UPDATE' -c 'SELECT timescaledb_pre_restore()'"
ssh airsunbox-ec2 "sudo docker exec -i iot_db pg_restore -U iot_user -d iot_platform --no-
```

```
owner" < airsunbox-AAAA-MM-DD.dump
ssh airsunbox-ec2 "sudo docker exec iot_db psql -U iot_user -d iot_platform -c 'SELECT
timescaledb_post_restore()'"
ssh airsunbox-ec2 "sudo docker start iot_backend"
```

`ALTER EXTENSION … UPDATE` iguala la versión de TimescaleDB de la base nueva (que hereda la de la plantilla) con la del respaldo; sin ese paso `timescaledb_post_restore()` falla por *catalog version mismatch*. Procedimiento probado el 2026-10-27: 560 514 lecturas restauradas sin errores.

5.3 Disco completo

El disco raíz se puede volver a un *snapshot* sin detener la instancia (conserva la IP):

```
aws ec2 create-replace-root-volume-task --region us-east-2 \
  --instance-id i-07a637df8369c3f6c --snapshot-id <snap-id>
```

Se pierde todo lo escrito después del snapshot. Respaldo vigente del 2026-10-27: `snap-08db3f398cc7ebf28`.

5.4 Secretos

SSM guarda el historial de versiones de cada parámetro:

```
aws ssm get-parameter-history --region us-east-2 --name /airsunbox/prod/JWT_SECRET \
  --query 'Parameters[].[Version,LastModifiedDate]' --output table
```

Para volver a una versión anterior:

1. Leerla con `get-parameter --name /airsunbox/prod/X:<versión> --with-decryption`.
2. Guardarla con `put-parameter --overwrite`.
3. Recrear el backend (sección 7, I-02).

En el caso de `DB_PASSWORD`, la base también tiene que tener ese valor.

6. Monitoreo

6.1 Chequeos

Qué	Comando	Esperado
Web	`curl -s -o /dev/null -w '%{http_code}' https://18.223.121.208/`	`200`
API y base	`curl -s https://18.223.121.208/api/public/reports/stats`	JSON con totales
Contenedores	`ssh airsunbox-ec2 "sudo docker ps --format '{{.Names}}\t{{.Status}}'"`	5 `Up`, con `healthy` donde aplica
Errores del backend	`ssh airsunbox-ec2 "sudo docker logs --since 1h iot_backend 2>&1 | grep -c '\"level\":50'"`	0
Secretos cargados	`ssh airsunbox-ec2 "sudo docker logs iot_backend 2>&1 | grep 'Secretos cargados'"`	`JWT_SECRET, DB_PASSWORD, SMTP_PASS`
Disco	`ssh airsunbox-ec2 "df -h / && sudo docker system df"`	Uso de / < 80 %
Certificado	`echo | openssl s_client -connect 18.223.121.208:443 2>/dev/null | openssl x509 -noout -enddate`	Vence en ≥ 1 día (certificados de 6 días, renovación cada 12 h)
Renovación	`ssh airsunbox-ec2 "systemctl list-timers | grep certbot"`	Próxima ejecución programada
Estaciones	`ssh airsunbox-ec2 "sudo docker exec iot_db psql -U iot_user -d iot_platform -c 'select name, last_seen from devices order by last_seen desc nulls last limit 10'"`	`last_seen` reciente en las estaciones activas
Métricas	`ssh airsunbox-ec2 "sudo docker exec iot_backend wget -qO- localhost:3000/metrics | head -40"`	Métricas Prometheus
CI/CD	GitHub → Actions	Último `CI` y `deploy` en verde

6.2 Frecuencia sugerida

- **Después de cada despliegue:** web, contenedores, errores.
- **Diario:** disco, certificado, estaciones con datos.
- **Semanal:** respaldo de la base (sección 5.2) y revisión del historial de Actions.

Pendiente de mejora: alarmas automáticas en CloudWatch (CPU, disco, estado de la instancia) y un *uptime check* externo.

7. Atención de incidentes

Para cada incidente: confirmar el síntoma, diagnosticar con los comandos indicados, aplicar la solución, verificar con la sección 2.1 y registrarlo en la sección 9.

I-01 · La web no responde (código 000, 502 o 504)

Diagnóstico:

- `ssh airsunbox-ec2 "sudo docker ps -a --format '{{.Names}}\t{{.Status}}'"`.
- Si el SSH tampoco responde: revisar el estado de la instancia en la consola de EC2 (*status checks*).

Solución:

- **Docker detenido:** `ssh airsunbox-ec2 "sudo systemctl start docker"`. Los contenedores tienen `restart: unless-stopped` y vuelven solos.
- **iot_frontend caído:** `cd $APP && sudo docker compose -p airsunbox -f docker-compose.yml up -d --no-deps frontend`.
- **Backend caído:** ver I-02.
- **La instancia no responde:** reiniciarla desde la consola (*Reboot*, **nunca *Stop***).

I-02 · El backend no arranca o queda `unhealthy`

Diagnóstico:

```
ssh airsunbox-ec2 "sudo docker logs --tail 80 iot_backend 2>&1"
```

Mensaje en el log	Causa	Solución
`No se pudieron leer los secretos de SSM … faltan: …`	Sin acceso a SSM (rol, red o parámetro borrado)	Verificar el parámetro con `aws ssm get-parameters --names /airsunbox/prod/...` y la política `airsunbox-leer-secretos` del rol `sanik-iot-ec2-role`
`DB_PASSWORD no está definido` / `password authentication failed`	La contraseña de SSM no coincide con la de Postgres	Volver a aplicarla con `infra/seguridad/2-rotar-db-y-limpiar.sh` o con la versión anterior (sección 5.4)
`JWT_SECRET no está definido`	Falta el parámetro en SSM	Restaurarlo (sección 5.4)

Mensaje en el log	Causa	Solución
ECONNREFUSED … 5432 / 6379	Base o Redis caídos	Ver I-03

Para recrear el backend:

```
cd $APP && sudo docker compose -p airsunbox -f docker-compose.yml up -d --no-deps --force-recreate backend
```

I-03 · Base de datos o Redis caídos, o disco lleno

Diagnóstico:

- `ssh airsunbox-ec2 "df -h / && sudo docker logs --tail 50 iot_db"`.

Solución:

- **Disco lleno:**
  1. Liberar espacio con `sudo docker image prune -f` y `sudo docker builder prune -f`.
  2. Revisar el tamaño de `dots`. La compresión de TimescaleDB ya está activa.
  3. Si hace falta, ampliar el volumen EBS (*Modify volume*) y extender la partición (`growpart` + `resize2fs`).
- **Contenedor caído:** `cd $APP && sudo docker compose -p airsunbox -f docker-compose.yml up -d timescaledb redis`.
- **Datos dañados:** restaurar el último respaldo (sección 5.2).

I-04 · El navegador avisa que el certificado no es válido

Diagnóstico:

- Ver la fecha de vencimiento (sección 6.1, certificado).
- `ssh airsunbox-ec2 "sudo certbot renew --dry-run"`.

Solución:

1. Renovar: `sudo certbot renew`.
2. Recargar nginx: `sudo docker exec iot_frontend nginx -s reload`.
3. Comprobar que exista el *hook* `/etc/letsencrypt/renewal-hooks/deploy/recargar-nginx.sh`, que hace esa recarga automáticamente en cada renovación.

I–05 · No llegan los correos (2FA o recuperación de contraseña)

Diagnóstico:

```
ssh airsunbox-ec2 "sudo docker logs --since 1h iot_backend 2>&1 | grep -iE 'smtp|correo|mail'
| tail -20"
```

Solución:

- **Invalid login o 535:** la contraseña de aplicación de Gmail venció o fue revocada.
  1. Generar una nueva en la cuenta de Google del remitente.
  2. Guardarla en `/airsunbox/prod/SMTP_PASS`.
  3. Recrear el backend.
- **El correo sale pero no llega:** revisar Spam y el límite diario de envíos de Gmail.

I–06 · Estaciones sin datos o "0 estaciones activas"

Diagnóstico:

- `last_seen` de las estaciones (sección 6.1).
- `ssh airsunbox-ec2 "sudo docker logs --since 30m iot_backend 2>&1 | grep -i dots | tail"`.

Solución:

- Una estación cuenta como activa si envió datos en los **últimos 15 minutos** (`ONLINE_WINDOW` en `backend/src/lib/deviceStatus.ts`). Si envía con menos frecuencia, aparecerá inactiva entre envíos.
- **404 Token inválido en POST /api/dots/...:** el token de la estación no coincide. Copiar el token desde el detalle de la estación al firmware.
- **No hay ninguna petición:** el problema está del lado del equipo (WiFi, energía, URL del firmware).

I–07 · El job `deploy` falla

Leer el log del job en GitHub Actions (las credenciales aparecen ocultas).

Error	Solución
`test … branch --show-current = main` falla	En el servidor: `git checkout main`
`test -z "$(git status --porcelain)"` falla	Hay cambios locales en el servidor: revisarlos con `git status` y descartarlos o moverlos fuera del repositorio
`Backend health check failed`	Ver I-02
Error de OIDC o `AccessDenied`	Revisar el rol `airsunbox-github-deploy` y la variable `AWS_DEPLOY_ROLE_ARN` del repositorio

I-08 · Sospecha de un secreto filtrado

1. **Rotarlo de inmediato:**
   - **Contraseña de la base:** `infra/seguridad/2-rotar-db-y-limpiar.sh`.
   - **`JWT_SECRET`:** generar uno nuevo y guardarlo en SSM (`put-parameter --overwrite`). Al recrear el backend, **todas las sesiones se cierran.**
   - **`SMTP_PASS`:** revocar la contraseña de aplicación en Google y crear una nueva.
2. Recrear el backend y verificar (sección 2.1).
3. Si el secreto llegó al repositorio: quitarlo del código. El historial de git lo conserva, y por eso la rotación es obligatoria.
4. **Claves de AWS o contraseñas de usuarios IAM expuestas:** desactivarlas en IAM y revisar CloudTrail.

I-09 · Muchos usuarios ven "Tu contraseña cambió" o quedan fuera de sesión

Causa: se cambió `JWT_SECRET`, o hubo cambios de contraseña (`SESSION_REVOKED`).

Solución: es el comportamiento esperado. Los usuarios vuelven a iniciar sesión. No requiere intervención.

I-10 · Respuestas 429 (*Too Many Requests*)

Causa: el límite de peticiones por IP (10 por minuto en login, 2FA y recuperación).

Solución:

- Esperar un minuto.

- Si afecta a usuarios legítimos detrás de un mismo proxy, revisar `TRUST_PROXY` en `backend/.env`.

I-11 · La instancia se detuvo o cambió la IP

La instancia no tiene Elastic IP: al detenerla y volver a encenderla, AWS le asigna otra IP pública. Con la IP vieja dejan de funcionar:

- el certificado (emitido para la IP);
- `FRONTEND_URL` en `backend/.env` y los orígenes de CORS (`backend/src/app.ts`);
- `frontend/nginx.conf` (`server_name`, redirección y rutas del certificado);
- la verificación del job `deploy` (`--resolve` en el workflow);
- el alias SSH `airsunbox-ec2`;
- el firmware de las estaciones.

Encender la instancia:

```
aws ec2 start-instances --region us-east-2 --instance-ids i-07a637df8369c3f6c
aws ec2 wait instance-running --region us-east-2 --instance-ids i-07a637df8369c3f6c
aws ec2 describe-instances --region us-east-2 --instance-ids i-07a637df8369c3f6c \
  --query "Reservations[].Instances[].PublicIpAddress" --output text
```

Solución permanente:

1. Asignar una **Elastic IP** a la instancia (gratis mientras esté encendida), para que la dirección no vuelva a cambiar.
2. Emitir un certificado para la nueva IP (o, mejor, para un dominio).
3. Actualizar `FRONTEND_URL`, los orígenes de CORS y `frontend/nginx.conf` por PR.
4. Actualizar el workflow (`--resolve`), el alias SSH y el firmware de las estaciones.

8. Mantenimiento

Tarea	Frecuencia	Cómo
Respaldo de la base	Semanal y antes de migraciones	Sección 5.2
Rotación de la contraseña de la base	Semestral o ante sospecha	`infra/seguridad/2-rotar-db-y-limpiar.sh`
Revisión de accesos IAM y MFA	Mensual	Consola IAM: usuarios con MFA, sin claves sin uso
Limpieza de imágenes Docker	Mensual	`sudo docker image prune -f`
Borrar respaldos del cifrado del disco	Una vez confirmada la estabilidad	Volumen `vol-04b76514fb273997c` y snapshot `snap-08db3f398cc7ebf28`
Actualizar dependencias	Mensual	PR con `npm update`; el CI valida

9. Registro de cambios operativos

Fecha	Cambio	Impacto
2026-10-26	Despliegue automático por GitHub Actions + SSM (OIDC)	Sin pasos manuales
2026-10-26	Cabeceras de seguridad HTTPS y cierre de sesión en el servidor	Ninguno
2026-10-26	Contraseña de la base rotada; secretos movidos a SSM Parameter Store (KMS)	Segundos (reinicio del backend)
2026-10-27	Primer intento de cifrado del disco: fallo por la descripción no ASCII del snapshot; Docker quedó detenido	~10 min fuera de servicio; script corregido para levantar Docker ante error
2026-10-27	Disco raíz cifrado (`vol-0aaf1e03561f5d951`, KMS `aws/ebs`); cifrado EBS por defecto activado	~15 min planificados; IP conservada
2026-10-29	Instancia detenida al cerrar la fase 3, para ahorrar créditos de AWS	Plataforma fuera de línea hasta encenderla (I-11)